

BCSE0556 — Hadoop & Big Data Analytics

Time: 3 Hours | Max Marks: 50

Unit	Topic	Exam %
Unit 1	Big Data Concepts & Digital Data Types	90-100%
Unit 2	HBase & Cassandra (NoSQL)	90-100%
Unit 3	HDFS, YARN & Apache Spark	85-95%
Unit 4	Apache Pig & Apache Hive	90-100%
Unit 5	Kafka, Streaming & Real-World Systems	70-80%
Unit 6	Data Scientist & ML in Big Data	60-70%
+	HiveQL Quick Reference	CONFIRM
+	Pig Latin Quick Reference	CONFIRM
+	Cassandra CQL Quick Reference	CONFIRM

UNIT 1

Big Data Concepts & Digital Data Types

CO1 | 90-100% Exam Probability

1.1 What is Big Data?

Big Data refers to extremely large datasets that cannot be processed by traditional data processing tools due to their size, complexity, and speed of generation. It requires distributed computing frameworks like Hadoop to store and analyze such data.

1.2 The 5 V's of Big Data (MUST MEMORIZE)

V	Name	Definition	Example
V1	Volume	Massive amount of data generated every second	Facebook: 500TB/day
V2	Velocity	Speed at which data is generated and processed	Twitter: 6000 tweets/sec
V3	Variety	Different types and formats of data	Text, Video, Audio, XML, JSON
V4	Veracity	Uncertainty/trustworthiness of data	Fake news, noisy sensor data
V5	Value	Useful insights extracted from data	Fraud detection, recommendations

1.3 Types of Digital Data (CONFIRM — Both PYQs)

Three Types — MUST KNOW WITH EXAMPLES

1. Structured Data — Organized in rows and columns, fixed schema.

- Student table in RDBMS (rollno, name, marks)
- E-commerce order with fixed fields (customer name, price, address)
- CSV/Excel files with defined columns

2. Unstructured Data — No predefined format, cannot fit in rows/columns.

- CCTV footage / drone video recordings (binary video files)
- Audio books as MP3 files
- Images taken from mobile camera
- Zoom meeting recordings

3. Semi-Structured Data — Has some structure (tags/markers) but not rigid.

- Email (has To, From, Subject fields but body is free text)
- Facebook/Twitter posts (has timestamp, hashtags but text is free)
- XML files
- JSON data

1.4 Hadoop Ecosystem Components

Tool	Role / Purpose
HDFS	Hadoop Distributed File System — stores data across multiple nodes
MapReduce	Programming model for distributed data processing
YARN	Yet Another Resource Negotiator — manages cluster resources
Hive	SQL-like query language (HiveQL) for Big Data analysis
Pig	High-level scripting language (Pig Latin) for data transformation
HBase	NoSQL column-family database on top of HDFS
Sqoop	Transfers data between HDFS and RDBMS (import/export)
Zookeeper	Distributed coordination service — leader election, config management
Kafka	Distributed message broker for real-time data streaming
Flume	Collects and ingests log data into HDFS
Spark	In-memory distributed computing engine, 100x faster than MapReduce

UNIT 2

HBase & Cassandra (NoSQL)

CO2 | 90-100% Exam Probability

2.1 What is HBase?

HBase is an open-source, distributed, column-oriented NoSQL database built on top of HDFS. It is modeled after Google's Bigtable and designed to handle billions of rows and millions of columns. HBase provides real-time read/write access to Big Data.

2.2 HBase Architecture Components (DRAW THIS)

Component	Role
HMaster	Coordinates RegionServers. Handles schema changes, load balancing, failover. Like NameNode in HDFS.
RegionServer	Handles read/write requests for a region. Stores data in MemStore and HFiles.
Region	A subset of table rows stored contiguously. Tables split into multiple regions automatically.
MemStore	In-memory write buffer. Data written here first, then flushed to HFile on disk.
HFile	On-disk storage file on HDFS. Immutable, sorted key-value store.
Zookeeper	Keeps track of HMaster, RegionServers. Handles distributed coordination and leader election.
WAL (Write Ahead Log)	Logs all changes before writing to MemStore. Used for crash recovery.

2.3 HBase Data Model

Key Terms

- **Row Key** — Unique identifier for each row. Design critically for performance.
- **Column Family** — Group of related columns defined at table creation (e.g., "personal:", "marks:").
- **Column** — Individual field within a column family (e.g., "personal:name").
- **Cell** — Intersection of row + column + timestamp. Contains actual value.
- **Timestamp** — Version identifier. HBase stores multiple versions of each cell.

2.4 HBase vs RDBMS vs HDFS

Feature	HBase	RDBMS	HDFS
Data Model	Column Family	Tables/Rows	Files/Blocks
Schema	Flexible (schema-less)	Fixed schema	No schema
Joins	Not supported	Supported	Not applicable
Transactions	Row-level only	Full ACID	Not applicable

Best For	Random read/write	Structured queries	Sequential read
Scalability	Horizontal (commodity)	Vertical (expensive)	Horizontal

2.5 Apache Cassandra

Cassandra is a distributed NoSQL database designed for handling large amounts of data across many commodity servers with no single point of failure. It uses a **peer-to-peer ring architecture** (masterless) unlike HBase which has a master.

Key Cassandra Concepts:

- **Keyspace** — Top-level namespace (like a database in SQL). Contains tables.
- **Table** — Collection of rows with a schema. Defined in keyspace.
- **Primary Key** — Partition Key + Clustering Key. Determines data distribution.
- **Replication Factor** — Number of copies of data stored across nodes (default=3).
- **Consistency Level** — How many nodes must respond: ONE, QUORUM, ALL.
- **Hinted Hand-off** — If a node is down, another stores a "hint" and delivers the write when the node recovers. Ensures no data loss.
- **Partitioner** — Uses consistent hashing to distribute data across ring nodes.

2.6 Cassandra Collection Data Types

Three Collection Types (CO6 — appeared in PYQ)

- **List** — Ordered collection of elements (allows duplicates). Example: list<text> for phone numbers.
- **Set** — Unordered collection of unique elements. Example: set<text> for tags/hashtags.
- **Map** — Key-value pairs. Example: map<text, int> for subject-marks pairs.

UNIT 3

HDFS, YARN & Apache Spark

CO3 | 85-95% Exam Probability

3.1 HDFS Architecture

HDFS (Hadoop Distributed File System) is the primary storage layer of Hadoop. It splits files into large blocks (default **128 MB**) and distributes them across multiple DataNodes. It follows a **write-once, read-many** model.

Component	Role
NameNode	Master node. Stores metadata (file names, block locations, permissions). Does NOT store actual data. Single point of failure.
DataNode	Slave nodes. Store actual data blocks. Send heartbeats to NameNode every 3 seconds. Multiple DataNodes for fault tolerance.
Secondary NameNode	NOT a backup NameNode! Periodically merges fsimage and edits log to create a checkpoint. Reduces NameNode startup time.
Block	Default size = 128 MB. Files split into blocks distributed across nodes.
Replication	Default replication factor = 3. Same block stored on 3 different nodes (rack-aware placement).

3.2 YARN Architecture

YARN (Yet Another Resource Negotiator) is the resource management layer of Hadoop 2.x. It separates resource management from data processing, allowing multiple applications to share the cluster.

- **Resource Manager (RM)** — Global master. Accepts job submissions, allocates resources, schedules containers. Has Scheduler + Application Manager.
- **Node Manager (NM)** — Per-machine agent. Manages containers on that node. Monitors CPU/RAM usage. Sends heartbeats to RM.
- **Application Master (AM)** — Per-application process. Negotiates resources from RM, coordinates with NMs to execute tasks.
- **Container** — Allocated unit of resources (CPU + RAM) on a node. Tasks execute inside containers.

■ Node Manager Crash — YARN Recovery (PYQ 2024-25)

- NM sends regular heartbeats to RM.
- If RM stops receiving heartbeats → NM marked as dead.
- RM notifies all Application Masters running on that NM.
- Application Masters request new containers from RM on healthy nodes.
- Tasks that were running on crashed NM are rescheduled on other nodes.
- Completed tasks are NOT re-run (output already saved to HDFS).
- YARN Schedulers: FIFO, Capacity Scheduler, Fair Scheduler.

3.3 Apache Spark vs MapReduce (CONFIRM)

Feature	Apache Spark	Hadoop MapReduce
---------	--------------	------------------

Processing	In-memory (RAM)	Disk-based (HDFS)
Speed	100x faster for iterative	Slower due to disk I/O
Execution Model	DAG (Directed Acyclic Graph)	Only Map → Reduce
Data Abstraction	RDD, DataFrame, Dataset	Key-Value pairs only
Real-time	Spark Streaming supported	Batch processing only
Fault Tolerance	RDD lineage graph	Data replication on HDFS
ML Support	MLlib built-in	Mahout (external)
Language	Scala, Python, Java, R	Java primarily
Use Case	Iterative ML, streaming, SQL	Simple batch ETL jobs

RDD (Resilient Distributed Dataset) — Fundamental data structure of Spark. Immutable, distributed collection of objects. Supports transformations (map, filter, groupBy) and actions (count, collect, save). Fault-tolerant via lineage graph.

DAG (Directed Acyclic Graph) — Spark builds a DAG of all transformations before executing. Optimizes execution by pipelining stages and minimizing shuffles.

UNIT 4

Apache Pig & Apache Hive

CO4 | 90-100% Exam Probability

4.1 Apache Pig — Overview

Apache Pig is a high-level data flow scripting platform for processing large datasets on Hadoop. It uses **Pig Latin**, a procedural language. Pig Latin scripts compile down to MapReduce jobs automatically.

Philosophy of Apache Pig:

- "Pigs eat anything" — Pig can process any type of data (structured, semi-structured, unstructured).
- "Pigs live anywhere" — Can run on any environment (local, Hadoop cluster).
- "Pigs are domestic animals" — Easy to control and use.
- "Pigs fly" — Can process data at high speed and scale.

4.2 Pig Modes & Execution Modes (PYQ — Both Papers)

Mode Type	Mode	Description
Running Mode	Local Mode	Runs on single JVM. Uses local file system. For testing. Command: pig -x local
	MapReduce Mode	Default mode. Runs on Hadoop cluster with HDFS. Command: pig (no flag)
	Tez Mode	Uses Apache Tez DAG engine. Faster than MapReduce. Command: pig -x tez
Execution Mode	Interactive (Grunt)	Pig Latin statements run one by one in Grunt shell. Type: pig → grunt>
	Batch Mode	Run a complete .pig script file. Command: pig script.pig
	Embedded Mode	Pig Latin embedded inside Java/Python programs.

4.3 Pig Latin Data Model (PYQ — Define + Diagram)

Term	Definition
Atom	A single atomic value. e.g., "John", 25, 8.5
Field	A data item in a tuple. Can be any data type.
Tuple	Ordered set of fields (like a row). e.g., (1, "John", 8.5)
Bag	Collection of tuples (like a table). Can have duplicate tuples. e.g., {(1,"A"),(2,"B")}
Relation	A bag with a name. The outermost data structure in Pig. Like a table in relational databases.

4.4 Pig vs Hive vs MapReduce

Feature	Pig	Hive	MapReduce
Language	Pig Latin (procedural)	HiveQL (SQL-like, declarative)	Java code
User	Data engineers/ETL developers	Analysts familiar with SQL	Java programmers
Schema	Not required at load time	Required (table schema)	Not required

Type	Data flow language	Data warehouse / query	Low-level API
OLAP	Not designed for OLAP	Designed for OLAP	Not designed for OLAP
Partitioning	Not native	Native (static + dynamic)	Not native
Speed	Faster than raw MR	Similar to Pig	Slowest

4.5 Apache Hive — Overview

Hive is a data warehouse infrastructure built on Hadoop. It provides an SQL-like interface (HiveQL) to query and analyze structured data stored in HDFS. Hive is designed for **OLAP (Online Analytical Processing)** — NOT for OLTP.

Hive Partitioning:

- **Why partitioning?** — Improves query performance by scanning only relevant partition instead of entire table.
- **Static Partitioning** — User manually specifies partition value during data loading. Faster but less flexible.
- **Dynamic Partitioning** — Hive automatically determines partition value from data. Requires: SET `hive.exec.dynamic.partition=true`;

4.6 Hive Applications:

- ETL (Extract, Transform, Load) pipelines
- Log analysis (web server logs, clickstream data)
- Business intelligence and reporting dashboards
- Ad-hoc queries on Big Data stored in HDFS
- Data summarization, aggregation, and trend analysis
- NOT suitable for: real-time queries, OLTP, low-latency operations

UNIT 5

Kafka, Streaming & Real-World Systems

CO3/CO6 | 70-80% Exam Probability

5.1 Apache Kafka Architecture

Kafka is a distributed, fault-tolerant, high-throughput **publish-subscribe messaging system**. Used for building real-time data pipelines and streaming applications.

- **Producer** — Publishes (sends) messages to a Kafka topic.
- **Consumer** — Subscribes to a topic and reads messages from it.
- **Topic** — Logical channel to which messages are published. Like a category or feed name.
- **Partition** — Topics are divided into partitions for parallelism. Each partition is an ordered, immutable sequence.
- **Broker** — Kafka server that stores messages and serves consumers. A Kafka cluster has multiple brokers.
- **Consumer Group** — Multiple consumers sharing partitions of a topic for parallel consumption.
- **Offset** — Position of a consumer in a partition. Tracks what has been read.
- **Zookeeper** — Used by Kafka for distributed coordination and broker management.

5.2 Real-Time Fraud Detection Architecture (PYQ Case Study)

Architecture Flow (Write this in exam)

Data Ingestion: Kafka receives transaction events from payment gateway in real-time. Each transaction is a Kafka message in "transactions" topic.

Stream Processing: Spark Streaming / Flink consumes from Kafka. Applies rules: velocity check (3 txns in 500ms = fraud), amount threshold, location mismatch.

Stateful Storage: HBase stores user purchase history, velocity counters per user. Redis for fast in-memory state lookup.

Decision Engine: If fraud score exceeds threshold → block transaction, send alert to Kafka "fraud-alerts" topic.

Response: Payment gateway subscribes to "fraud-alerts" topic and rejects transaction before authorization. All within ~500ms.

5.3 Sqoop & Zookeeper

- **Sqoop** — Tool to **import** data from RDBMS (MySQL, Oracle) to HDFS and **export** data from HDFS back to RDBMS. Uses MapReduce internally. Command: `sqoop import --connect jdbc:mysql://...`
- **Zookeeper** — Distributed coordination service. Provides: leader election, distributed locking, configuration management, naming registry. Used by HBase, Kafka, YARN.
- **Hinted Hand-off** (Cassandra) — When a target node is down, a coordinator node stores the write as a "hint". When the target node recovers, the hint is replayed to ensure consistency.
- **Peer-to-Peer Network** — No central master. All nodes are equal (e.g., Cassandra ring). vs Master-Slave (e.g., HDFS, HBase).
- **NoSQL** — Not Only SQL. Databases that store data in non-tabular formats: Key-Value (Redis), Column (Cassandra, HBase), Document (MongoDB), Graph (Neo4j).

UNIT 6

Data Scientist & ML in Big Data

CO5/CO6 | 60-70% Exam Probability

6.1 Role of Data Scientist

A Data Scientist extracts meaningful insights from large datasets using statistics, machine learning, and domain knowledge to help organizations make data-driven decisions.

Key Skills:

- Statistics & Mathematics
- Machine Learning (Supervised/Unsupervised)
- Programming: Python, R, Scala
- Big Data: Hadoop, Spark, Hive
- Data Visualization: Tableau, Power BI
- SQL and NoSQL databases
- Deep Learning: TensorFlow, PyTorch
- Domain expertise (Finance, Healthcare)

Example: Netflix Data Scientist analyzes viewing history of 200 million users using Spark MLlib to build a recommendation system that suggests personalized content, increasing engagement by 35%.

6.2 ML Algorithms in Big Data (CO5)

Algorithm	Type	How it works	Big Data Use Case
Linear Regression	Supervised	Fits a line $y=mx+b$ to predict continuous output	Predict house prices, stock values
Logistic Regression	Supervised	Predicts probability using sigmoid function for binary classification	Spam detection, fraud detection
K-Means Clustering	Unsupervised	Groups data into K clusters based on nearest centroid	Customer segmentation, anomaly detection
Decision Tree	Supervised	Tree structure of if-else rules. Splits data on best feature value	Disease diagnosis, loan approval
Random Forest	Supervised	Ensemble of multiple Decision Trees. Majority vote decision	Feature selection, fraud detection

Memorize these exact commands. They appear in EVERY paper.

Database Operations

- **Create database:**

```
CREATE DATABASE bigdata;
```

- **List all databases:**

```
SHOW DATABASES;
```

- **Show database details:**

```
DESCRIBE DATABASE bigdata;
```

- **Use/Mount database:**

```
USE bigdata;
```

- **Drop database:**

```
DROP DATABASE bigdata;
```

Table Operations

- **Create managed table:**

```
CREATE TABLE btech_bigdata (  
rollno INT,  
name STRING,  
branch STRING,  
gpa FLOAT  
);
```

- **Create external table:**

```
CREATE EXTERNAL TABLE emp (  
id INT, name STRING  
)  
LOCATION '/user/hive/data/';
```

- **Show all tables:**

```
SHOW TABLES;
```

- **Describe table structure:**

```
DESCRIBE btech_bigdata;
```

- **Drop table:**

```
DROP TABLE btech_bigdata;
```

Data Loading & Querying

- **Load data (local):**

```
LOAD DATA LOCAL INPATH '/root/hivedemo/students.tsv'  
INTO TABLE btech_bigdata;
```

- **Load data (HDFS):**

```
LOAD DATA INPATH '/hdfs/students.tsv'  
INTO TABLE btech_bigdata;
```

- **Select all rows:**

```
SELECT * FROM btech_bigdata;
```

- **Select specific column:**

```
SELECT name FROM btech_bigdata;
```

- **Filter with condition:**

```
SELECT name, gpa FROM btech_bigdata  
WHERE gpa > 4.8;
```

- **Count rows:**

```
SELECT COUNT(*) FROM btech_bigdata;
```

- **Order results:**

```
SELECT * FROM btech_bigdata  
ORDER BY gpa DESC;
```

- **Group by:**

```
SELECT branch, COUNT(*) FROM btech_bigdata  
GROUP BY branch;
```

Partitioning

- **Create partitioned table:**

```
CREATE TABLE emp_part (  
id INT, name STRING, salary FLOAT  
)  
PARTITIONED BY (dept STRING);
```

- **Enable dynamic partition:**

```
SET hive.exec.dynamic.partition = true;  
SET hive.exec.dynamic.partition.mode = nonstrict;
```

- **Load into partition (static):**

```
LOAD DATA LOCAL INPATH '/data/cse.csv'  
INTO TABLE emp_part PARTITION (dept='CSE');
```

- **Insert into dynamic partition:**

```
INSERT INTO TABLE emp_part PARTITION (dept)  
SELECT id, name, salary, dept FROM emp;
```

Hive CLI Commands

- **Change to HIVE_HOME:**

```
cd $HIVE_HOME
```

- **Start Hive CLI:**

```
hive
```

- **Run script file:**

```
hive -f script.hql
```

- **Exit Hive CLI:**

```
EXIT; or QUIT;
```

• LOAD — Load data from HDFS

```
employees = LOAD '/data/emp.csv' USING PigStorage(',') AS  
(emp_id:int, name:chararray, dept:chararray, salary:float);
```

• FILTER — Filter rows by condition

```
rich = FILTER employees BY salary > 60000;
```

• GROUP — Group rows by a field

```
grp = GROUP employees BY dept;
```

• FOREACH...GENERATE — Transform and compute

```
result = FOREACH grp GENERATE group, COUNT(employees);  
-- group = dept name, COUNT = number of employees in each dept
```

• ORDER — Sort data

```
sorted = ORDER employees BY salary DESC;
```

• DISTINCT — Remove duplicate rows

```
unique = DISTINCT employees;
```

• JOIN — Join two relations

```
joined = JOIN employees BY dept, dept_table BY dept_id;
```

• LIMIT — Limit output rows

```
top5 = LIMIT employees 5;
```

• STORE — Save output to HDFS

```
STORE result INTO '/output/dept_count' USING PigStorage(',');
```

• DUMP — Print output to console (testing)

```
DUMP result;
```

• DESCRIBE — Show schema of a relation

```
DESCRIBE employees;
```

• Complete Example — Count employees per dept

```
employees = LOAD '/data/emp.csv' USING PigStorage(',') AS  
(id:int, name:chararray, dept:chararray, salary:float);  
high_sal = FILTER employees BY salary > 60000;  
grp = GROUP high_sal BY dept;  
count = FOREACH grp GENERATE group AS dept, COUNT(high_sal) AS cnt;  
sorted = ORDER count BY cnt DESC;  
STORE sorted INTO '/output/result' USING PigStorage(',');
```

• Create Keyspace (GLA_University example)

```
CREATE KEYSPACE GLA_University
WITH REPLICATION = {
'class': 'SimpleStrategy',
'replication_factor': 3
};
```

• Use a Keyspace

```
USE GLA_University;
```

• Create Table with Primary Key

```
CREATE TABLE CSE_Dept (
RollNo INT PRIMARY KEY,
Name TEXT,
Course TEXT,
Branch TEXT,
CPI FLOAT
);
```

• Insert Data

```
INSERT INTO CSE_Dept (RollNo, Name, Course, Branch, CPI)
VALUES (1001, 'Arjun', 'B.Tech', 'CSE-DA', 8.5);
```

• Select All Rows

```
SELECT * FROM CSE_Dept;
```

• Select with Condition

```
SELECT Name, CPI FROM CSE_Dept WHERE RollNo = 1001;
```

• Update Data

```
UPDATE CSE_Dept SET CPI = 9.0 WHERE RollNo = 1001;
```

• Delete Data

```
DELETE FROM CSE_Dept WHERE RollNo = 1001;
```

• Show All Tables in Keyspace

```
DESCRIBE TABLES;
-- or
SELECT table_name FROM system_schema.tables
WHERE keyspace_name = 'gla_university';
```

• Show All Keyspaces

```
DESCRIBE KEYSPACES;
```

• Import CSV into Cassandra Table

```
COPY CSE_Dept (RollNo, Name, Course, Branch, CPI)
FROM '/data/students.csv'
WITH DELIMITER=',' AND HEADER=TRUE;
```

• Export Cassandra Table to CSV

```
COPY CSE_Dept TO '/output/students_export.csv'
WITH DELIMITER=',' AND HEADER=TRUE;
```

- **Drop Table / Keyspace**

```
DROP TABLE CSE_Dept;
```

```
DROP KEYSPACE GLA_University;
```

UNIT ★

Last-Minute Cheatsheet — Read 1 Hour Before Exam

Most Important Points

★ Big Data 5 V's

- Volume (size)
- Velocity (speed)
- Variety (types)
- Veracity (trust)
- Value (insights)

★ **Structured = RDBMS/CSV | Unstructured = Video/Audio/Image | Semi-structured = JSON/XML/Email**

★ HDFS

- NameNode = metadata master
- DataNode = stores 128MB blocks
- Secondary NN ≠ backup
- Replication = 3
- Write-once read-many

★ YARN

- Resource Manager = global scheduler
- Node Manager = per-node agent
- App Master = per-job coordinator
- Container = unit of CPU+RAM

★ Spark vs MapReduce

- Spark = in-memory = 100x faster
- Spark = DAG engine
- MapReduce = disk-based batch
- RDD = immutable distributed collection

★ Pig Modes

- Local(-x local), MapReduce(default), Tez(-x tez)
- Interactive(Grunt shell), Batch(.pig file)
- LOAD→FILTER→GROUP→FOREACH→STORE

★ Hive

- HiveQL = SQL-like
- OLAP not OLTP
- Static partition = manual
- Dynamic = auto
- CREATE→LOAD→SELECT→WHERE

★ HBase

- Column-family NoSQL on HDFS
- HMaster + RegionServer + MemStore + HFile
- Zookeeper for coordination
- Row Key design is critical

★ Cassandra

- Masterless peer-to-peer ring
- Hinted Hand-off = no data loss on node failure
- Replication Factor = copies
- Consistency: ONE < QUORUM < ALL
- CREATE KEYSPACE→CREATE TABLE→INSERT→SELECT

★ Kafka

- Producer→Topic→Consumer
- Partition = unit of parallelism
- Broker = Kafka server
- Offset = consumer position
- Zookeeper manages brokers

■ All the best for your End Term Exam 2026!

Focus on: HiveQL + Pig Latin + HBase + Cassandra CQL + Hadoop Ecosystem

These 5 topics = 35-40 marks guaranteed.